رواد مصر الرقمية

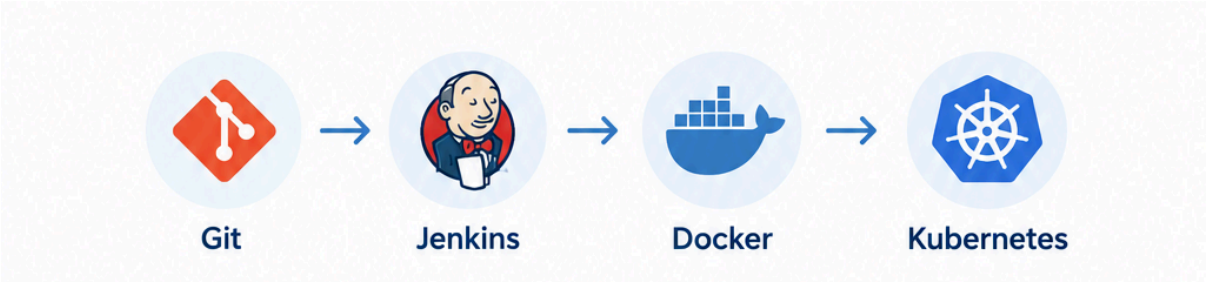
وزارة الاتصالات وتكنولوجيا المعلومات

DEPI
Digital Egypt Pioneers Initiative

DEVOPS TRACK
Final Project Documentation

AUTOMATED E-COMMERCE DEPLOYMENT PLATFORM

A DevOps-Based Solution for Automated, Scalable, and Reliable E-Commerce Deployment



SUBMITTED TO
Eng. Mena Magdy

PREPARED BY:

- Eng. Moataz Diaa
- Eng. Youssef Ahmed
- Eng. Omar Ahmed
- Eng. Mona Ayman
- Eng. Rahma AbdElalim

SUBMISSION DATE
16 October 2026

ACADEMIC YEAR
2026–2027

AUTOMATE • DEPLOY • SCALE • MONITOR

TABLE OF CONTENTS

Project Documentation

1. **Project Planning & Management**
    - 1.1 Project Proposal
    - 1.2 Project Objectives
    - 1.3 Project Scope
    - 1.4 Project Plan and Timeline
    - 1.5 Milestones and Deliverables
    - 1.6 Task Assignment and Roles
    - 1.7 Resource Allocation
    - 1.8 Risk Assessment and Mitigation Plan
    - 1.9 Key Performance Indicators (KPIs)
2. **Literature Review**
    - 2.1 Introduction
    - 2.2 DevOps and Continuous Integration/Continuous Delivery (CI/CD)
    - 2.3 Containerization and Orchestration
    - 2.4 Infrastructure as Code
    - 2.5 Monitoring and Reliability
    - 2.6 Auto-Scaling and Automated Rollback
    - 2.7 Feedback and Evaluation
    - 2.8 Suggested Improvements
    - 2.9 Final Grading Criteria
3. **Requirements Gathering**
    - 3.1 Stakeholder Analysis
    - 3.2 User Stories
    - 3.3 Use Cases
    - 3.4 Functional Requirements
    - 3.5 Non-Functional Requirements

PART 1: PROJECT PLANNING & MANAGEMENT

1.1 Project Proposal

1.1.1 Project Overview

Project Title: Automated E-Commerce Deployment Platform

The Automated E-Commerce Deployment Platform is a DevOps project that aims to automate the deployment and management of a simplified multi-service e-commerce application. It integrates CI/CD, containerization, infrastructure automation, monitoring, and recovery mechanisms to improve deployment consistency, scalability, and reliability.

The proposed platform uses Git, Jenkins, Docker, Kubernetes, Terraform, Ansible, Nginx, Prometheus, Grafana, and selected AWS services where required.

1.1.2 Problem Statement

Traditional deployment processes often involve manual configuration, repetitive tasks, and inconsistent environments. These challenges can cause deployment failures, downtime, and difficulties managing interconnected application services.

A structured DevOps workflow is therefore needed to automate deployments, monitor application health, manage infrastructure consistently, and support recovery from failed updates.

1.1.3 Proposed Solution

The project will implement a CI/CD workflow in which Jenkins retrieves code from Git, runs the required tests, builds Docker images, and deploys validated application versions to Kubernetes.

Terraform and Ansible will support infrastructure provisioning and configuration management. Nginx will handle traffic routing, while Prometheus and Grafana will provide monitoring and visualization. A rollback procedure will be tested to support recovery from unsuccessful deployments.

1.1.4 Expected Outcome

The expected outcome is a maintainable deployment platform that reduces manual effort, improves deployment consistency, supports scaling, and provides visibility into application performance and deployment health.

1.2 Project Objectives

The main objective is to implement an automated, scalable, and reliable deployment workflow for a multi-service e-commerce application.

The specific objectives are to:

1. Automate build, testing, and deployment through Jenkins CI/CD pipelines.
2. Containerize application services using Docker.
3. Deploy and scale containerized workloads using Kubernetes.
4. Provision selected infrastructure using Terraform.
5. Automate applicable server configuration using Ansible.
6. Configure Nginx for reverse-proxy routing.
7. Monitor application and infrastructure metrics using Prometheus and Grafana.
8. Implement and test a rollback procedure.
9. Evaluate deployment duration, pipeline success, response time, scaling, and recovery.
10. Maintain source code, configurations, and documentation in a Git repository.

Expected Result: A tested DevOps workflow demonstrating automated deployment, infrastructure management, monitoring, and recovery.

1.3 Project Scope

1.3.1 In-Scope Activities

The project includes:

- Developing or integrating a simplified e-commerce application with a web interface, product catalog/search, order handling, and database integration.
- Simulating payment functionality for demonstration purposes, if required.
- Creating Dockerfiles and container configurations.
- Implementing Jenkins CI/CD pipelines.
- Deploying application services to Kubernetes and configuring health checks and horizontal scaling where supported.
- Preparing Terraform configurations for selected AWS resources, such as EC2, RDS, S3, and a load balancer, according to budget and architecture.
- Creating Ansible playbooks for applicable configuration tasks.
- Configuring Nginx reverse-proxy routing.
- Integrating Prometheus and Grafana.
- Testing deployment, monitoring, scaling, and rollback.
- Maintaining project documentation and a Git repository.

1.3.2 Out-of-Scope Activities

The initial scope excludes a production-scale commercial marketplace, real payment processing, storage of payment-card information, AI-based recommendations, multi-region disaster recovery, and enterprise-level compliance systems.

These features may be considered future improvements if time and resources permit.

1.3.3 Assumptions and Constraints

The project depends on the team's available time, skills, computing resources, and budget. AWS usage depends on account access, permissions, and estimated costs.

The application and infrastructure may be simplified according to technical feasibility while preserving the core DevOps objectives.

1.3.4 Main Scope Deliverables

The project is expected to produce:

- A simplified e-commerce application.
- Dockerfiles and container configurations.
- Jenkins pipeline definitions.
- Kubernetes manifests and scaling configurations.
- Terraform files and Ansible playbooks.
- Nginx configuration.
- Prometheus configuration and Grafana dashboards.
- A tested rollback procedure.
- A Git repository, test evidence, and technical documentation.

1.4 Project Plan and Timeline

1.4.1 Project Plan Overview

The project will follow six proposed phases over six weeks. The schedule is preliminary and may be adjusted according to the actual deadline, team availability, and technical challenges.

1.4.2 Project Timeline

Phase	**Main Activities**	**Duration**	**Expected Output**
1. Planning	Scope, requirements, roles, and risks	Week 1	Planning documentation
2. Application Development	Implement the basic application and services	Week 2	Application prototype

3. Containerization	Create and test Docker images	Week 3	Containerized application
4. CI/CD and Kubernetes	Configure Jenkins and deploy workloads	Week 4	Pipeline and deployment
5. Infrastructure and Monitoring	Configure Terraform, Ansible, Nginx, Prometheus, and Grafana	Week 5	Infrastructure and monitoring setup
6. Testing and Finalization	Test scaling and rollback, resolve issues, and finalize documentation	Week 6	Tested platform and final report

1.4.3 Gantt Chart

Activity	**W1**	**W2**	**W3**	**W4**	**W5**	**W6**
Planning and Requirements	■					
Application Development		■	■			
Docker Containerization			■			
Jenkins CI/CD				■		
Kubernetes Deployment				■	■	
Terraform and Ansible					■	

Nginx Configuration					■	
Monitoring Setup					■	■
Testing and Rollback						■
Documentation and Presentation	■				■	■

Legend: ■ = *Planned activity period.*

1.4.4 Progress Monitoring

The team will track tasks using a shared task list, review progress regularly, maintain configurations in Git, and record test results. Delays and technical blockers will be discussed and addressed by adjusting priorities or reallocating tasks.

1.5 Milestones and Deliverables

1.5.1 Project Milestones

ID	**Milestone**	**Target**	**Completion Criteria**
M1	Planning and Requirements	Week 1	Scope, requirements, roles, and risks documented
M2	Application and Containers	Weeks 2–3	Application services run successfully in Docker
M3	CI/CD Pipeline	Week 4	Jenkins automates the agreed build and deployment stages
M4	Kubernetes Deployment	Weeks 4–5	Application deployed and scaling configuration tested

M5	Infrastructure and Monitoring	Week 5	Infrastructure configuration and monitoring prepared
M6	Testing and Rollback	Week 6	Core tests and rollback scenarios documented
M7	Final Submission	Weeks 5–6	Report, repository, test evidence, and presentation reviewed

1.5.2 Deliverables and Acceptance Criteria

Deliverable	Acceptance Criteria
E-commerce Application	Agreed core services operate in the selected environment
Docker Configuration	Images build and containers run successfully
Jenkins Pipeline	Configured pipeline stages execute and produce logs
Kubernetes Configuration	Workloads deploy successfully and scaling is tested where supported
Terraform and Ansible	Configuration files are validated and execution results documented
Nginx	Requests are routed to the intended application endpoints
Monitoring	Prometheus collects selected metrics and Grafana displays them

Rollback	A previous working version is restored successfully in planned tests
Git Repository	Required source code, configuration, and setup instructions are available
Final Report and Presentation	Documentation and test evidence are reviewed and organized

Deliverables will be accepted based on demonstrated functionality and recorded evidence, not merely on the existence of configuration files. Features that cannot be tested must be identified with their limitations.

1.6 Task Assignment and Roles

1.6.1 Team Structure

The project team consists of five members with proposed responsibilities distributed as follows.

Team Member	**Proposed Role**	**Main Responsibilities**
Rahma	Project Coordinator and Documentation Lead	Coordinate tasks, track milestones, maintain documentation, and organize reviews
Youssef	CI/CD and Automation Engineer	Configure Jenkins, automate build and testing stages, and integrate Git
Mona	Application and Containerization Engineer	Develop or integrate application services and prepare Docker configurations
Omar	Kubernetes and Networking Engineer	Configure Kubernetes deployments, scaling, and Nginx routing

Moataz	Infrastructure and Monitoring Engineer	Prepare Terraform and Ansible configurations and support Prometheus and Grafana

1.6.2 Task Allocation

Task	Primary Owner	Support
Planning and requirements	Rahma	All members
Application development	Mona	Youssef
Docker and container testing	Mona	Youssef
Git repository and Jenkins	Youssef	All members
Kubernetes and scaling	Omar	Youssef, Moataz
Nginx configuration	Omar	Mona
Terraform and Ansible	Moataz	Omar, Youssef
Monitoring dashboards	Moataz	Omar
Integration and rollback testing	Youssef	Omar, Mona, Moataz
Documentation and presentation	Rahma	All members

1.6.3 Collaboration

The team will use Git for version control, a shared task list for progress tracking, and regular meetings to discuss completed tasks and blockers. Members will review important changes and participate in integration testing to ensure that the components work together.

1.7 Resource Allocation

1.7.1 Human Resources

The five team members will contribute to planning, application development, containerization, CI/CD, Kubernetes, infrastructure automation, monitoring, testing, and documentation according to their assigned roles.

All members will participate in integration testing and final evaluation.

1.7.2 Software and Technical Resources

Resource	Purpose
Git/GitHub	Version control and collaboration
Docker	Application containerization
Jenkins	CI/CD automation
Kubernetes	Deployment and scaling
Terraform	Infrastructure provisioning
Ansible	Configuration management
Nginx	Reverse-proxy routing
Prometheus	Metrics collection

Grafana	Monitoring dashboards
Programming Framework and Database	Application implementation and data storage
Testing Tools	Functional and performance evaluation

The programming framework, database, and testing tools will be selected according to team skills and project requirements.

1.7.3 Infrastructure and Collaboration Resources

The project may use local computers, virtual machines, a local Kubernetes cluster, or AWS resources depending on technical requirements and budget.

Google Docs will support collaborative documentation, GitHub will host project files, and a shared task list will track responsibilities and deadlines.

1.7.4 Resource Constraints and Cost Management

The team will prioritize essential project features, estimate cloud costs before provisioning, monitor resource usage, and remove unused resources. Local or free tools will be preferred when they satisfy the project requirements.

Not all listed AWS services must be deployed; the final selection will depend on the agreed architecture and available budget.

1.8 Risk Assessment and Mitigation Plan

1.8.1 Introduction

Risk assessment helps the team identify technical, financial, scheduling, and coordination problems that may affect project delivery. Risks will be reviewed throughout implementation, and mitigation measures will be applied when necessary.

1.8.2 Risk Assessment Matrix

ID	Risk	Likelihood	Impact	Mitigation Strategy

R1	Integration issues between DevOps tools	Medium	High	Integrate components incrementally and test each stage
R2	Limited experience with some technologies	High	Medium	Use official documentation and allocate learning time
R3	CI/CD or Kubernetes deployment failures	Medium	High	Validate configurations, inspect logs, and test deployments
R4	Unexpected cloud costs or resource limitations	Medium	High	Monitor usage, estimate costs, and remove unused resources
R5	Exposure of credentials or secrets	Medium	High	Keep secrets out of Git and restrict access
R6	Monitoring gaps or missing metrics	Medium	Medium	Verify Prometheus targets and Grafana data
R7	Downtime or unsuccessful deployment	Medium	High	Use health checks, controlled rollouts, and tested rollback
R8	Data loss or database connectivity issues	Low	High	Use test data, verify connections, and maintain backups where applicable
R9	Uneven workload or communication problems	Medium	Medium	Assign clear owners and review progress regularly

R10	Insufficient time for testing and documentation	Medium	High	Track milestones and prioritize essential deliverables

1.8.3 Risk Monitoring and Contingency Plan

The team will review risks regularly, validate configuration files, track changes in Git, monitor cloud costs, and document issues.

If a deployment fails, the team will inspect logs, correct the problem, and redeploy or restore the previous stable version. If resources or time become limited, the team will prioritize core features and use an alternative testing environment where appropriate.

1.8.4 Conclusion

Proactive risk management, early testing, secure configuration, and clear task ownership will help reduce avoidable delays and improve the reliability of the project.

1.9 Key Performance Indicators (KPIs)

1.9.1 Introduction

Key Performance Indicators are measurable metrics used to evaluate the deployment platform's automation, performance, availability, scaling, and recovery capabilities. The proposed targets below will be evaluated through testing.

1.9.2 Project Success Metrics

KPI	**Proposed Target**	**Measurement Method**
Deployment Duration	Within 5 minutes where feasible	Measure deployment start and successful completion times using Jenkins logs
CI/CD Pipeline Success Rate	At least 90%	Successful pipeline runs ÷ total completed runs × 100
Application Response Time	P95 of 500 ms or less under a defined test load	Measure HTTP response-time percentiles

Application Availability	At least 99% during the defined test period	Successful availability checks ÷ total scheduled checks × 100
Rollback Success Rate	100% of planned rollback tests	Verify that the previous version is restored and healthy
Auto-Scaling Effectiveness	Demonstrate scaling under controlled load	Inspect replica counts, resource metrics, and scaling events
Monitoring Coverage	All agreed critical services monitored	Verify Prometheus metrics and Grafana dashboards
Infrastructure Validation	Planned validation tests complete without unresolved errors	Review Terraform and Ansible execution results

1.9.3 KPI Evaluation Process

The team will define test conditions, establish baseline measurements, run deployment and performance tests, collect logs and dashboard screenshots, and compare the results against the proposed targets. Any unmet target will be documented with the relevant limitations and corrective actions.

1.9.4 Success Criteria

The project will be evaluated based on:

- Successful CI/CD execution and container deployment.
- Verified Kubernetes deployment and scaling behavior.
- Working monitoring dashboards and metric collection.
- Successful rollback tests.
- Documented performance and availability measurements.
- Organized infrastructure configurations, source code, and project documentation.

1.9.5 Conclusion

The KPIs provide a consistent way to evaluate the platform's effectiveness and identify opportunities to improve automation, performance, monitoring, scalability, and recovery.

PART 2: LITERATURE REVIEW

2.1 Introduction

DevOps is a software development and operations approach that emphasizes collaboration, automation, continuous integration, continuous delivery, and continuous monitoring. It aims to reduce manual work, improve software delivery speed, and increase system reliability.

The Automated E-Commerce Deployment Platform applies these principles to a simplified e-commerce application composed of multiple interconnected services. This section reviews the main technologies and practices that support the proposed solution.

2.2 DevOps and CI/CD

Continuous Integration (CI) involves frequently integrating code changes and validating them through automated builds and tests. Continuous Delivery or Deployment (CD) extends this process by preparing or automatically releasing validated changes to a target environment.

Jenkins can automate these stages by connecting the source-code repository with testing, image building, and deployment processes. This reduces repetitive manual tasks and helps identify errors earlier in the development lifecycle.

In this project, Jenkins and Git will provide the foundation for an automated and traceable software delivery workflow.

2.3 Containerization and Orchestration

Containerization packages an application and its dependencies into a consistent execution environment. Docker supports this approach by enabling application components to be built and distributed as container images.

For applications containing multiple services, Kubernetes provides orchestration capabilities such as workload management, service discovery, health checks, and scaling. These capabilities help manage application deployments more consistently.

The project will use Docker to package the selected e-commerce services and Kubernetes to deploy and manage them.

2.4 Infrastructure as Code and Configuration Management

Infrastructure as Code (IaC) allows infrastructure to be defined and managed through version-controlled configuration files rather than relying entirely on manual provisioning.

Terraform supports declarative infrastructure provisioning, while Ansible automates configuration tasks such as software installation and server setup. Both tools can improve consistency, repeatability, and maintainability when used with appropriate testing and access controls.

In this project, Terraform and Ansible will support infrastructure provisioning and configuration management according to the selected deployment architecture.

2.5 Monitoring and Reliability

Monitoring provides visibility into application health, resource utilization, and operational problems. Prometheus collects and stores time-series metrics, while Grafana presents metrics through dashboards and visualizations.

Monitoring helps the team detect unhealthy services, investigate performance issues, and evaluate the effectiveness of deployment and scaling configurations.

The proposed platform will use Prometheus and Grafana to monitor selected application and infrastructure metrics.

2.6 Reverse Proxy, Auto-Scaling, and Rollback

A reverse proxy receives incoming requests and forwards them to appropriate backend services. Nginx can perform this function and provide a controlled entry point for application traffic.

Kubernetes horizontal auto-scaling can adjust the number of application replicas according to configured metrics and resource conditions. Automated rollback procedures can restore a previous application version when a deployment fails, provided that the recovery process is configured and tested correctly.

These practices will be used to demonstrate traffic routing, workload scaling, and recovery from deployment failures.

2.7 Feedback and Evaluation

Feedback and evaluation help determine whether the project meets its intended objectives and academic requirements.

The team will use supervisor or lecturer feedback, implementation reviews, testing evidence, and project milestones to identify weaknesses and improve the solution. Feedback will be recorded when it is received and incorporated into the project where appropriate.

Lecturer's Assessment: The lecturer's actual assessment and comments will be added after they are provided.

2.8 Suggested Improvements

The project may be improved through the following activities:

- Strengthening automated testing and code-quality checks in the CI/CD pipeline.
- Improving security through appropriate secrets management and restricted access.
- Adding alerts for service failures and abnormal resource utilization.
- Expanding performance testing under different workloads.
- Improving rollback validation and documenting recovery procedures.
- Optimizing cloud resource usage and reviewing infrastructure costs.
- Extending the monitoring dashboards to cover additional service-level indicators.

These improvements may be prioritized according to the project timeline, available resources, and feedback received.

2.9 Final Grading Criteria

The final grading criteria depend on the official assessment rubric provided by the lecturer or course. Until that rubric is available, the team should evaluate its work against the following proposed categories.

Assessment Category	Evidence to Prepare
Documentation	Clear project report, requirements, architecture, and technical explanations.
Implementation	Working application services, Docker configuration, CI/CD pipeline, and deployment setup.
Automation and Infrastructure	Validated Jenkins, Kubernetes, Terraform, and Ansible configurations, as applicable.
Monitoring and Reliability	Prometheus and Grafana evidence, scaling tests, and rollback validation.
Testing and Evaluation	Test cases, recorded results, identified limitations, and corrective actions.
Presentation and Demonstration	Clear explanation of the architecture, tool integration, project results, and team contributions.

PART 3: REQUIREMENTS GATHERING

3.1 Stakeholder Analysis

3.1.1 Introduction

Stakeholder analysis identifies the individuals and groups who are involved in, affected by, or interested in the Automated E-Commerce Deployment Platform. Understanding their needs helps define the system requirements and ensures that the proposed solution supports both application users and the technical team responsible for deployment and maintenance.

3.1.2 Identified Stakeholders

Stakeholder	Role and Interest	Main Requirements
Customers	Use the e-commerce application to browse products and place orders.	Easy navigation, product search, reliable order submission, and clear feedback.
System Administrator	Manages application data and operational settings where applicable.	Controlled access, manageable services, and reliable system operation.
Developers	Develop and update application code.	Version control, automated testing, consistent development environments, and clear build results.
DevOps Engineers	Manage CI/CD, deployment, infrastructure, and monitoring.	Automated pipelines, container management, infrastructure automation, monitoring, and rollback capabilities.
Project Supervisor	Reviews project progress and evaluates the submitted work.	Clear documentation, demonstrable functionality, testing evidence, and progress reports.

Project Team	Designs, implements, tests, and documents the platform.	Clear responsibilities, integration between tools, manageable tasks, and measurable objectives.

3.1.3 Stakeholder Needs and Priorities

The main stakeholder priorities are summarized as follows:

- **Customers:** Reliable access to the application and a straightforward shopping experience.
- **Developers:** A consistent process for submitting and validating code changes.
- **DevOps Engineers:** Repeatable deployments, infrastructure consistency, visibility into system health, and recovery from failed releases.
- **Administrators:** Manageable application services and appropriate access control.
- **Project Supervisor:** Evidence that the project requirements and objectives have been addressed.

3.1.4 Conclusion

The stakeholder analysis establishes the main needs that guide the requirements of the platform. These needs will be translated into user stories, use cases, functional requirements, and non-functional requirements.

3.2 User Stories

3.2.1 Introduction

User stories describe system capabilities from the perspective of the people who interact with the application or manage its deployment. They help the project team understand what users need and why each capability is important.

The following user stories define the main expected interactions with the proposed platform.

3.2.2 Customer User Stories

US-01: Browse Products

As a customer, I want to browse the available products so that I can explore the items offered by the e-commerce application.

Acceptance Criteria:

- The application displays the available products.

- Each product displays its relevant information.
- The user receives a clear message if no products are available.

US-02: Search Products

As a customer, I want to search for products by name or keyword so that I can find items that match my interests.

Acceptance Criteria:

- The application accepts a search term.
- Matching products are displayed.
- A clear message appears when no matching products are found.

US-03: Place an Order

As a customer, I want to submit an order so that I can complete the main shopping workflow.

Acceptance Criteria:

- The application validates the submitted order information.
- Valid orders are recorded in the system.
- The customer receives an order confirmation or an appropriate error message.

US-04: View Order Status

As a customer, I want to view the status of my order so that I can understand whether it has been received or processed.

Acceptance Criteria:

- The application displays the order status when the feature is implemented.
- Customers can access only the orders they are authorized to view.

3.2.3 Developer and DevOps User Stories

US-05: Automate Code Validation

As a developer, I want code changes to trigger an automated pipeline so that errors can be detected before deployment.

Acceptance Criteria:

- A configured repository event triggers the Jenkins pipeline.
- The pipeline retrieves the relevant source code.
- Configured tests execute and their results are recorded.
- Failed validation prevents the release from proceeding to deployment.

US-06: Build Versioned Container Images

As a DevOps engineer, I want the pipeline to build and tag Docker images so that application versions can be identified and deployed consistently.

Acceptance Criteria:

- The image is built using the defined Docker configuration.
- Images receive identifiable version tags.
- Image publishing occurs when a registry is configured and the required checks pass.

US-07: Deploy and Scale the Application

As a DevOps engineer, I want to deploy containerized services to Kubernetes and configure horizontal scaling so that the application can respond to changing workloads.

Acceptance Criteria:

- The configured workloads can be deployed successfully.
- Health checks are configured where applicable.
- Scaling behavior is tested using supported metrics and a defined workload.

US-08: Monitor System Health

As a DevOps engineer, I want to view application and infrastructure metrics so that I can identify performance issues and unhealthy services.

Acceptance Criteria:

- Prometheus collects the selected metrics.
- Grafana displays the configured metrics in dashboards.
- Missing metrics or unhealthy services can be investigated using available monitoring information.

US-09: Recover from a Failed Deployment

As a DevOps engineer, I want to restore a previous stable application version when a deployment fails so that service disruption can be reduced.

Acceptance Criteria:

- Deployment health is checked after release.
- The failure condition is identified.
- The configured rollback procedure restores a previous version when supported.
- The result of the recovery attempt is recorded.

US-10: Manage Infrastructure Consistently

As a DevOps engineer, I want infrastructure and server configuration to be defined through version-controlled files so that environments can be created and configured consistently.

Acceptance Criteria:

- Terraform configuration files define the selected infrastructure resources.
- Ansible playbooks define applicable configuration tasks.
- Validation and execution results are documented.

3.3 Use Cases

3.3.1 Introduction

Use cases describe how actors interact with the system to achieve specific goals. They provide a structured view of the application's main functions and the deployment workflow.

3.3.2 Main Actors

Actor	Description
Customer	Browses products, searches the catalog, and submits orders.
Administrator	Manages application data or settings where these functions are implemented.
Developer	Submits code changes to the source-code repository.
Jenkins CI/CD Pipeline	Automates configured testing, image building, and deployment stages.
DevOps Engineer	Configures the deployment environment, monitors system health, and handles failures.

3.3.3 Use Case UC-01: Browse and Search Products

Primary Actor: Customer

Preconditions:

- The e-commerce application is available.
- Product data is accessible.

Main Flow:

1. The customer opens the e-commerce application.
2. The system retrieves and displays available products.
3. The customer enters a search term if needed.
4. The system processes the search request.
5. The system displays matching products or a message indicating that no matches were found.

Alternative Flow:

- If the product service or database is unavailable, the system displays an appropriate error message when error handling is implemented.

Postconditions:

- The customer can view the available products or the search results.

3.3.4 Use Case UC-02: Place an Order

Primary Actor: Customer

Preconditions:

- The customer can access the application.
- The required product and order functionality is available.

Main Flow:

1. The customer selects the desired product or products.
2. The customer submits the required order information.
3. The system validates the order.
4. The system records the valid order.
5. The system displays a confirmation message.

Alternative Flow:

- If validation fails, the system requests corrected information.
- If order processing fails, the system displays an error and avoids reporting the order as successfully completed.

Postconditions:

- A valid order is recorded, or the customer is informed that the order could not be completed.

Note: Payment processing may be simulated for the project demonstration. Real payment-card processing is outside the initial project scope.

3.3.5 Use Case UC-03: Automated CI/CD Deployment

Primary Actor: Developer

Supporting Components: Git, Jenkins, Docker, container registry where configured, and Kubernetes.

Preconditions:

- The source-code repository is configured.
- Jenkins has the required pipeline and permissions.
- The required build tools and deployment environment are available.

Main Flow:

1. The developer pushes code changes to the Git repository.
2. The configured repository event triggers Jenkins.
3. Jenkins retrieves the source code.
4. The pipeline runs the configured validation and testing stages.
5. If the tests pass, Jenkins builds and tags the Docker image.
6. The image is published to the configured registry, if required.
7. The pipeline deploys the selected application version to Kubernetes.
8. The deployment health is checked.
9. The pipeline records the outcome.

Alternative Flow:

- If testing fails, the pipeline stops before deployment.
- If image building or publishing fails, the pipeline reports the failure and does not proceed through dependent stages.
- If deployment or health verification fails, the configured recovery procedure is executed where supported.
- The pipeline records the failure for investigation.

Postconditions:

- The new version is deployed and verified, or the pipeline reports a failure and follows the configured recovery procedure.

3.3.6 Use Case UC-04: Monitor Application Health

Primary Actor: DevOps Engineer

Supporting Components: Prometheus and Grafana.

Preconditions:

- The monitoring configuration is available.
- The relevant metrics endpoints are accessible.

Main Flow:

1. Prometheus collects metrics from the configured targets.
2. Grafana retrieves the relevant monitoring data.
3. The DevOps engineer opens the dashboard.
4. The engineer reviews resource usage, application metrics, and service health indicators.
5. The engineer investigates abnormal values or missing metrics.

Alternative Flow:

- If a target is unavailable, the engineer checks the target status, connectivity, and monitoring configuration.

Postconditions:

- The engineer has access to the available monitoring information and can investigate detected issues.

3.3.7 Use Case UC-05: Roll Back a Failed Deployment

Primary Actor: DevOps Engineer or configured deployment pipeline.

Preconditions:

- A previous stable application version exists.
- A rollback mechanism has been configured.
- The deployment failure condition can be detected.

Main Flow:

1. A new application version is deployed.
2. Health verification detects a deployment failure.
3. The pipeline or engineer initiates the configured rollback procedure.
4. The system restores the previous version where supported.
5. The deployment health is checked again.
6. The result is recorded.

Alternative Flow:

- If rollback fails, the pipeline reports the recovery failure and the engineer investigates the deployment and cluster state.

Postconditions:

- The previous version is restored and verified, or the recovery failure is documented for further action.

3.4 Functional Requirements

3.4.1 Introduction

Functional requirements specify the operations and services that the system must provide. Each requirement below describes a capability expected from the proposed platform.

3.4.2 Application Functional Requirements

FR-01: Product Display

The system shall display available products and their relevant information.

FR-02: Product Search

The system shall allow customers to search for products using supported search terms.

FR-03: Order Management

The system shall validate and record valid customer orders and provide an appropriate confirmation or error message.

FR-04: Service Containerization

The application services selected for the project shall be packaged and executed using Docker containers.

3.4.3 CI/CD and Deployment Requirements

FR-05: Automated Pipeline Execution

Jenkins shall execute the configured CI/CD pipeline when a supported repository event occurs.

FR-06: Automated Validation and Testing

The pipeline shall run the configured validation and test stages and record their results.

FR-07: Docker Image Building and Versioning

The pipeline shall build Docker images and assign identifiable version tags to application releases.

FR-08: Image Publishing

Where a container registry is configured, the pipeline shall publish validated images using the required credentials and permissions.

FR-09: Kubernetes Deployment

The platform shall deploy the selected containerized application workloads to Kubernetes using version-controlled deployment configuration.

FR-10: Horizontal Scaling

The platform shall support horizontal scaling for selected workloads using a configured scaling mechanism and supported metrics.

FR-11: Health Verification

The deployment workflow shall verify application health using the configured health checks or validation procedures.

FR-12: Deployment Recovery

The platform shall provide a configured and testable rollback procedure for supported deployment failures.

3.4.4 Infrastructure and Networking Requirements

FR-13: Infrastructure Provisioning

Terraform shall define the selected infrastructure resources required by the final deployment architecture.

FR-14: Configuration Management

Ansible shall automate applicable server setup and configuration tasks where required by the selected environment.

FR-15: Reverse-Proxy Routing

Nginx shall route incoming requests to the intended application endpoints according to the configured routing rules.

3.4.5 Monitoring Requirements

FR-16: Metrics Collection

Prometheus shall collect the selected metrics from configured and accessible monitoring targets.

FR-17: Monitoring Dashboards

Grafana shall display relevant application and infrastructure metrics through configured dashboards.

FR-18: Operational Logs and Results

The team shall retain relevant pipeline logs, deployment outcomes, and test results to support troubleshooting and project evaluation.

3.4.6 Version Control and Documentation Requirements

FR-19: Version-Controlled Configuration

Application source code, pipeline definitions, and applicable deployment and infrastructure configuration files shall be maintained in the project Git repository.

FR-20: Technical Documentation

The project shall provide setup instructions, configuration explanations, testing evidence, and documented limitations.

3.5 Non-Functional Requirements

3.5.1 Introduction

Non-functional requirements define the quality attributes and operational constraints of the platform. They describe how well the system should operate in areas such as performance, reliability, scalability, security, and maintainability.

The targets below are proposed evaluation criteria. They must be verified through testing and should not be reported as achieved until actual results have been recorded.

3.5.2 Performance Requirements

NFR-01: API Response Time

The target 95th-percentile API response time should be no greater than 500 milliseconds under a defined test workload, where feasible.

NFR-02: Deployment Duration

The target deployment duration should be within five minutes for the agreed deployment scenario, where feasible. The measurement shall identify the start and end points used.

NFR-03: Resource Efficiency

The team shall monitor CPU and memory utilization during deployment and testing to identify resource constraints and opportunities for optimization.

3.5.3 Reliability and Availability Requirements

NFR-04: Availability Target

The target application availability should be at least 99% during a defined test period. The test duration, monitoring interval, and calculation method must be documented.

NFR-05: Health Detection

The platform shall use configured health checks or monitoring indicators to identify unhealthy application workloads.

NFR-06: Rollback Verification

The rollback procedure shall be tested under controlled failure scenarios. Recovery results and any limitations shall be documented.

3.5.4 Scalability Requirements

NFR-07: Horizontal Scaling

Selected Kubernetes workloads shall be configured to scale within defined minimum and maximum replica limits when the required metrics and scaling components are available.

NFR-08: Scaling Validation

Scaling behavior shall be tested under a controlled workload, and the observed replica changes and resource metrics shall be recorded.

3.5.5 Security Requirements

NFR-09: Secrets Protection

Passwords, access tokens, private keys, and other sensitive credentials shall not be committed to the public Git repository.

NFR-10: Access Control

Access to cloud resources, deployment environments, and CI/CD credentials shall be restricted according to the permissions required for each task.

NFR-11: Safe Demonstration Data

The project shall use test data for demonstrations and shall not store real payment-card information. Any payment functionality shall remain simulated unless an explicitly authorized integration is implemented.

NFR-12: Configuration Security

Infrastructure and application configurations shall follow appropriate security practices for the selected environment, including restricted network access where applicable.

3.5.6 Maintainability and Usability Requirements

NFR-13: Maintainable Configuration

Configuration files shall be organized, version-controlled, and documented so that team members can understand and update them.

NFR-14: Reproducibility

The project shall document the setup steps and dependencies required to reproduce the agreed deployment workflow.

NFR-15: Usability

The e-commerce interface shall provide understandable navigation, readable product information, and clear success or error messages for supported operations.

3.5.7 Monitoring and Testability Requirements

NFR-16: Monitoring Coverage

All agreed critical application components and relevant infrastructure targets shall have the monitoring coverage supported by the selected environment.

NFR-17: Test Evidence

The team shall record test conditions, observed results, and relevant logs or dashboard evidence for the main project requirements.

NFR-18: Measurable Evaluation

Performance, availability, pipeline success, scaling, and recovery shall be evaluated using documented measurement methods rather than unsupported claims.

3.5.8 Requirements Validation

The requirements will be validated through application tests, pipeline execution, deployment verification, monitoring checks, controlled scaling tests, and rollback scenarios. Each requirement should be marked as implemented, tested, partially implemented, or not implemented according to the actual project evidence.